

Neo4j & Knowledge Graphs

Across the Platform

Reference Documentation — K12, HP, G2G, EB, LMS+PAL

Prepared 2026-09-30

This document describes where Neo4j and knowledge-graph technology are used today across the platform's five product areas, how the underlying architecture works, and how the pieces fit together. It is written as reference documentation for anyone joining or reviewing this part of the system — not as an audit or issue log.

Contents

1. Overview
2. Scope
3. Modules & Features by Area
4. User Roles
5. Main Workflows
6. Technical Architecture
7. Database Structure
8. API Structure
9. Security
10. Current State

1. Overview

What is Neo4j?

Neo4j is a graph database. Instead of storing data as rows in tables joined by foreign keys, it stores nodes (things) and relationships (named, directed connections between things); both can carry properties. Its query language, Cypher, reads like a diagram of the pattern being searched for:

```
MATCH (c:Concept {id: 501})-[:REQUIRES]->(prereq:Concept)
RETURN prereq
```

That statement finds Concept 501, follows its REQUIRES relationships outward, and returns what is connected on the other end.

Relational databases handle “give me rows matching a condition” very well, but become expensive at “follow a chain of relationships an unknown number of hops deep.” A recursive SQL query for “everything Concept 501 depends on, transitively” needs a rewritten query per hop-depth. In Neo4j, the same question is `MATCH (c:Concept {id:501})-[:REQUIRES*1..5]->(x)` — the number of hops is a query parameter, not a structural change to the query.

Neo4j earns its keep specifically on multi-hop, variable-depth relationship questions — prerequisite chains, “what connects to what, transitively.” For single-table lookups and aggregates, which make up most day-to-day ERP work, plain SQL remains simpler and faster. That distinction is the working test used throughout this document for where Neo4j belongs.

Knowledge Graph, Graph Database, and Graph RAG — disambiguated

These three terms are related but distinct, and are often used interchangeably in casual conversation:

Term	What it is	Example from this platform
Graph database	A storage engine built around nodes and edges	Neo4j itself
Knowledge graph	A schema’d graph modeling real entities and their meaning — a data model, not a product	<code>(:Concept)-[:REQUIRES]->(:Concept), (:Question)-[:ASSESES]->(:Concept)</code>
Neo4j	The specific product implementing the graph database used here	<code>config/neo4j.php</code> , <code>GraphOutbox</code> , <code>GraphDrain</code> , <code>neo4j:*</code> artisan commands
Graph RAG	Using a knowledge graph as retrieval context for an LLM	Not implemented anywhere on the platform today — see Section 10

A knowledge graph is not a product or a piece of software — it is a data model. Neo4j is one way to store a knowledge graph. What makes a graph a “knowledge” graph rather than just “a graph” is that its relationships mean something domain-specific (REQUIRES, ASSESSES, TEACHES), not just “these two rows happen to be linked.”

Graph RAG is a retrieval technique: instead of retrieving similar text chunks for an LLM prompt (classic RAG, via a vector database), it retrieves a structured neighborhood from a knowledge graph — a node

plus its connected nodes — and hands that to the LLM as context. This platform has the knowledge graph half of that equation live in production (Section 3.5); it does not yet have a vector store, an embeddings pipeline, or any Graph RAG retrieval code. Where an LLM is used today (chapter-content and question generation via App\Domain\AI\Support\ModelClient, fronting Claude, DeepSeek, and Gemini/OpenRouter), it is prompt-in/completion-out only — not retrieval-augmented.

2. Scope

The platform spans five named product areas. Neo4j's presence across them is not one consistent strategy — it is three separate, only loosely related stories that happen to share the word “Neo4j”:

- The curriculum knowledge graph inside next_lms_erp — the platform's own sync pipelines feeding a live academic-hierarchy and prerequisite graph for K12 and LMS+PAL.
- HP's own, independently-built competency and job-role graph, live inside the hp_erp backend — unrelated code, unrelated pipelines, possibly (but not confirmedly) the same physical Neo4j server.
- A third-party-hosted skill-ontology viewer, embedded by iframe in two otherwise-unrelated frontends (G2G's and LMS+PAL's), which displays a graph but is not run, synced, or owned by any pipeline documented here.

These three are described separately throughout this document rather than merged into a single “graph strategy,” because treating them as one would misrepresent how the system actually works today.

At a glance

Area	Codebase(s)	Neo4j involvement today
K12	next_lms_erp (backend)	Partial — core academic hierarchy graph (Standard→Subject→Chapter→Concept), fed by the live outbox pipeline
HP	hp_erp (backend) + g2gv0 (frontend)	Yes, live — hp_erp's own competency / org / job-role graph; g2gv0 additionally embeds an external ontology viewer
G2G	next_lms_erp's G2G namespace	None — no Neo4j references in this namespace
EB	next_lms_erp (Brain/*)	None — dashboard/aggregation layer; its “knowledge-rag” widget is a plain SQL count, not retrieval
LMS+PAL	next_lms_erp (backend) + lms_k12 (frontend)	Yes, deepest — four sync pipelines, the Coherence Map, and (separately) an embedded external ontology viewer for capability/career features

One identity is shared underneath all five: sub_institute_id is a single tenant column used across every area — it identifies “which school,” not “which product.” Section 9 covers a real, intentional inconsistency in how the native tables and PAL's own tables each use this column for their shared/platform tenant.

3. Modules & Features by Area

3.1 K12 — the base school ERP

K12 is the core school ERP inside `next_lms_erp` — admissions, attendance, examination, fees, HR, library, transport, and the academic hierarchy that everything else is built on. Its Neo4j involvement is the academic-hierarchy graph: Standard, Subject, Chapter, and Concept nodes, connected by `HAS_SUBJECT` / `HAS_CHAPTER` / `HAS_CONCEPT` relationships. This hierarchy is the backbone that LMS+PAL's own graph (3.5) builds on top of, and it is fed by the same live outbox pipeline described in Section 6.

3.2 HP — HR and people-ops (`hp_erp` + `g2gv0`)

HP is a standalone system, not a namespace inside `next_lms_erp`: a Laravel backend (`hp_erp`) paired with a modern Next.js frontend (`g2gv0`), which authenticates against `hp_erp`'s own session and reads its database (`DB_NAME=hp_erp`). Its module set covers HR, payroll, talent management, leave, onboarding, and a school-sector LMS overlay.

Unlike every other area on the platform, HP has its own real, independently-built Neo4j integration — not an embed, not an absence:

- A dedicated `Neo4jService` (`app/Services/Neo4jService.php`) wraps the official `laudis/neo4j-php-client` library, authenticating from `NEO4J_URI` / `NEO4J_USERNAME` / `NEO4J_PASSWORD`.
- Three live graph controllers — `JobRoleGraphController`, `DepartmentGraphController`, `OrganizationGraphController` — run Cypher `MATCH` queries over `JobRole`, `Skill`, `Behaviour`, `Knowledge`, `Ability`, and `Attitude` nodes connected by `REQUIRES_*` relationships, and return the results as JSON.
- Two `Neo4jSyncController` implementations push LMS content rows into Neo4j — a sync mechanism entirely separate from `next_lms_erp`'s four pipelines (Section 6).
- Two distinct Bolt connections are configured: `NEO4J_URI` (the HR/competency graph) and a second, `NEO4J_LMS_URI`, whose port matches the one `next_lms_erp` itself uses for its own Neo4j. This suggests `hp_erp` may be configured to reach the same physical server that hosts the LMS+PAL curriculum graph — this is an observed configuration fact, not an independently confirmed shared-instance architecture, and should be treated as an open question until verified directly against both servers.

On the frontend side, `g2gv0` carries no Neo4j client of its own; its one touchpoint is a competency-taxonomy screen that embeds an externally-hosted graph viewer by `iframe` (see the cross-cutting note in 3.6). Whether that hosted viewer actually reads `hp_erp`'s own graph, or is a wholly separate Neo4j instance, is not established by either codebase.

3.3 G2G — people & competency, migrated

The G2G namespace inside `next_lms_erp` (`G2gLms/*`, `OrganizationManagement/*`, `TalentManagement/*`, `TaskManagement/*`, routed at `api/g2g-lms/*`) is a later, as-is migration of the same HR/people-ops domain into the main platform. It carries no Neo4j references of its own — the migration did not bring `hp_erp`'s graph integration with it. Its workloads are single-table lookups and

status aggregates (attendance counts, leave balances, task status), which is exactly the query shape where plain SQL is the right tool.

3.4 EB — Enterprise Brain

EB is a cross-module analytics and intelligence dashboard layer inside `next_lms_erp` (Brain/*, roughly two dozen Brain*IntelligenceController classes, served at `/api/brain`). It has no Neo4j involvement: everything it does is aggregation over SQL, which is SQL's strongest case, not a graph database's. One dashboard widget is keyed "knowledge-rag," but it is backed by plain SQL COUNT queries — a naming choice, not an implementation of retrieval-augmented generation.

3.5 LMS + PAL — the deep integration

The native Learning Management System (`app/Http/Controllers/lms/*`) and PAL, the adaptive-learning layer built on top of it (diagnostic, mastery, coherence, and the ESO decision engine), together carry the platform's deepest Neo4j integration: roughly 41 entity node-projections and 35 relationship specs, a live outbox sync, and a dedicated nightly bulk-projection pipeline for PAL's own derived structure. The full architecture is covered in Section 6, and its data model in Section 7.

Its flagship graph-backed feature is the Coherence Map — a prerequisite/post-requisite traversal that answers, for any concept, everything it depends on and everything that depends on it, to arbitrary depth, without looping on known reciprocal-pair cycles. It exists as two separate API surfaces and two separate frontend implementations, detailed in Section 8.

Separately from the curriculum graph, two newer areas under active development — capability-intelligence and career-intelligence — embed the same externally-hosted skill-ontology viewer used by HP/G2G's competency screen (see 3.6). This is a second, independent Neo4j touchpoint inside LMS+PAL — unrelated to the sync pipelines, the outbox, or the Coherence Map.

3.6 Cross-cutting: the shared external ontology viewer

One externally-hosted service, <https://skill-ontology-neo4j.vercel.app>, is embedded by `iframe` independently in two otherwise-unrelated frontends: `g2gv0`'s competency-taxonomy screen (`cm-taxonomy-ontology.tsx`) and `lms_k12`'s capability-intelligence screen (`taxonomy-ontology.tsx`). Neither frontend runs a Neo4j client itself — both simply display a page hosted elsewhere.

This is worth documenting as an open architectural question rather than a resolved fact: it is not confirmed whether this hosted viewer reads HP's own competency graph (3.2), a copy of it, or a wholly separate dataset; nor is its ownership or maintenance path established from either repository. Anyone extending either screen should treat this as something to verify directly rather than assume.

4. User Roles

Three roles interact with graph-backed features, each through a different surface:

Student / Learner

Reaches the graph indirectly, through the learner-facing Coherence Map inside PAL (lms_k12's app/pal/new screens). This view reads Neo4j directly via the /api/pal/coherence/* endpoints, showing prerequisite/post-requisite relationships for concepts the student is working through. The student never queries Neo4j themselves — the API and its cycle-safe traversal logic sit between them and the graph.

Teacher / Author

Uses a separate authoring Coherence Map (lms_k12's course-master screens) to review and edit prerequisite relationships between chapters while building curriculum. This screen deliberately reads MariaDB, not Neo4j — explained in Section 8 — so an author's edits are immediately reflected without waiting on a sync cycle. Teachers also indirectly benefit from graph-derived insight wherever weak-concept root-causing surfaces in class-level reporting.

Admin / Operations

Operates the Neo4j layer itself, not a graph-backed feature: running and monitoring the neo4j:* artisan command family (Section 8), reviewing sync health, and — on HP's side — the equivalent competency/org graph endpoints exposed by hp_erp's own controllers. No end-user-facing admin screen currently visualizes sync health; it is a command-line surface today.

5. Main Workflows

5.1 Live outbox sync (per-row, every minute)

This is the steady-state path by which an ordinary write to the SQL database of record becomes visible in the graph:

1. A business write happens (e.g. a question is re-tagged to a new concept).
2. A DB trigger on one of the whitelisted tables fires, inside the SAME transaction as the business write, writing a row to `sync_log` (for nodes) and/or `neo4j_sync_queue` (for relationships). Crash-safe and rollback-safe by construction, since both writes share the transaction.
3. `neo4j:drain` runs every minute, draining queued nodes first (an edge cannot attach to a node that does not exist yet), then relationships.
4. If a foreign key was re-pointed rather than newly set, the queue entry also carries the previous target, so the stale edge is deleted before the new one is written – the graph never accumulates duplicate edges for the same logical relationship.

5.2 Nightly PAL coherence projection (03:00)

PAL's own graph structure — prerequisite edges derived from expert-authored and AI-drafted source tables, misconception links, concept-node mastery links — does not map cleanly to “one row changed, one edge changed.” It is computed in a scheduled bulk pass instead: `pal:coherence-sync` runs nightly, scoped per tenant/standard/subject, executing eight projection steps in sequence to rebuild PAL's portion of the graph from its own source tables. A row whose `quality_status` flips to rejected, or that is deleted outright, has its corresponding graph edge retracted in the same pass — the projection does not just add, it reconciles.

5.3 A concrete traversal: diagnosing a wrong answer

Walking a real example end to end, using node and edge types that exist in the graph today:

1. Student answers Question 4521 incorrectly.
2. Question 4521 --[:ASSESES]--> Concept 501 (graph traversal)
3. Concept 501 --[:REQUIRES]--> Concept 340 (graph traversal)
4. The system checks the student's mastery of Concept 340
-> `learner_node_state.mastery_estimate` (read from SQL)
5. If mastery is low, content that TEACHES Concept 340 is recommended.
6. The student engages with that content; mastery is re-estimated.

Steps 2–3 are exactly the multi-hop pattern Neo4j is suited to, and are queryable today via the live ASSESSES/REQUIRES edges. Step 4 is answered from SQL directly: PAL's live diagnostic engine (EsoPolicyService, which drives Plan → Learn → Check → Mastery → Recall) reads `learner_node_state.mastery_estimate` and does not itself query Neo4j. The graph and the mastery data both exist independently; wiring the diagnostic step to consult the graph directly is a described, scoped opportunity rather than a rewrite (Section 10).

6. Technical Architecture

MySQL/MariaDB remains the system of record for the entire platform — all writes happen there, all transactions are guaranteed there. Neo4j is a read-optimized projection of a slice of that data, kept in sync so that graph-shaped questions can be answered without recursive SQL. It is a projection, not a migration: in principle, Neo4j can be rebuilt from MySQL at any time.

6.1 next_lms_erp: four sync pipelines

Four separate mechanisms, built at different times for different purposes, coexist inside next_lms_erp:

Pipeline	Mechanism	Cadence	Role
A — Live outbox	sync_log (nodes) + neo4j_sync_queue (relationships), DB triggers, drained by neo4j:drain	Per-row, every minute	Backbone of the K12 academic graph; ~41 entity types, ~35 relationship specs
B — PAL bulk projection	CoherenceGraphProjection.php, hand-written Cypher MERGE, run via pal:coherence-sync	Nightly, 03:00	PAL's derived prerequisite/misconception/mastery structure
C — uid batch loader	neo4j:export / load / verify, keyed on a composite uid	Manual, one-time per phase	A bulk-load approach for a universal node key; not the platform's primary path today
D — Legacy inline writer	Helper::neo4jCreateNode/Relationship, called from request code	Per-request	Predates the outbox pattern; superseded by Pipeline A

Config surfaces: config/neo4j.php (connection + the 44-table trigger whitelist under projections.triggered), config/neo4j_graph.php, and config/pal.php (PAL-specific projection settings). Both sync_log and neo4j_sync_queue are ordinary application tables managed through the same migration system as the rest of the schema.

Scheduling dependency: Laravel's schedule() registers what should run and when, but Laravel itself does not run anything on a timer — an OS-level scheduler (cron, or a Windows Scheduled Task calling schedule:run) has to actually invoke it on a recurring basis. On Windows, this additionally needs an S4U principal or an interactive php.exe console will pop every run. Confirming this is wired up on every deployment target is an operational checklist item, not a one-time setup step, since a schedule() entry with nothing running schedule:run is a silent no-op.

6.2 hp_erp: an independent, parallel graph

hp_erp does not use next_lms_erp's pipelines, config, or conventions — it has its own, entirely separate Neo4j wiring: the laudis/neo4j-php-client PHP driver, its own Neo4jService, its own pair of Neo4jSyncController implementations pushing LMS content rows into Neo4j, and its own two Bolt connection strings (Section 3.2). There is no shared code path between the two systems' graph layers — only a possible shared physical server, which is unconfirmed.

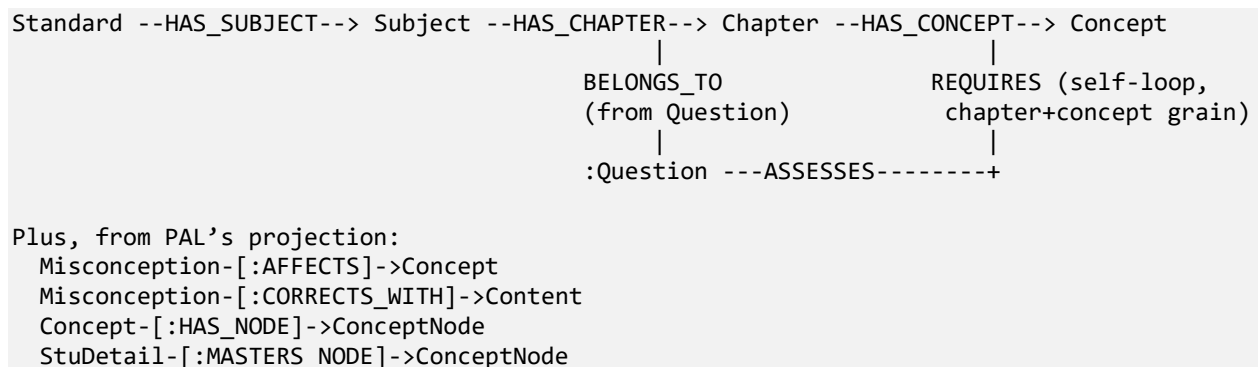
7. Database Structure

7.1 SQL side (next_lms_erp)

- sync_log — outbox table for node changes: table_name (the target Neo4j label), record_id, a JSON payload, and a status flag (e.g. PENDING).
- neo4j_sync_queue — outbox table for relationship changes: source_table/id, rel_type, target_table/id, and the previous target id (used to delete a stale edge when a foreign key is re-pointed).
- pal_* tables — PAL’s own sidecar tables (e.g. pal_concept_relations, pal_learning_relations, pal_misconception_library, pal_concept_nodes) feeding Pipeline B’s bulk projection; these are the source of truth the graph is derived from, not the graph itself.
- learner_node_state — SQL table holding mastery_estimate per student/concept-node; this is what PAL’s live diagnostic engine reads, not Neo4j.

7.2 Graph side — the curriculum hierarchy

The core structure synced into Neo4j by Pipeline A, extended by Pipeline B:



Unit and Topic are not labels in this graph — Pipeline B’s projection produces Subject→Chapter→Concept only, with no intermediate levels. This is why the authoring Coherence Map (Section 8) reads MariaDB directly: it needs a four-level hierarchy the graph does not yet have two of the levels for.

7.3 Storage direction vs. render direction

One convention is easy to get backwards, and doing so produces no error — only a curriculum that silently teaches in reverse:

- Storage (pal_concept_relations, pal_learning_relations): from = the node being learned, to = its prerequisite. Read a row as “from requires to.”
- Rendered graph (the Coherence Map’s output): source = the prerequisite, target = the dependent — so arrows point in the order things are learned.
- The write API sidesteps the ambiguity by naming roles explicitly — prerequisite_id / dependent_id — rather than ever using bare source/target.

The graph is also not required to be acyclic; a small number of reciprocal-pair cycles exist by design and are surfaced as warnings, never blocked.

7.4 Graph side — HP's competency graph

A structurally separate graph, live inside hp_erp: JobRole, Skill, Behaviour, Knowledge, Ability, and Attitude nodes, connected by `REQUIRES_*` relationships (e.g. a JobRole requiring a Skill, a Skill requiring underlying Knowledge). This models competency and job-role requirements for HR/talent workflows and shares no node labels or relationship types with the curriculum graph above.

8. API Structure

8.1 Two Coherence Map surfaces

The same feature name covers two deliberately separate API surfaces, serving two different audiences from two different data sources:

	Learner-facing	Authoring
Base path	/api/pal/coherence/*	/api/lms/coherence-map
Reads from	Neo4j	MariaDB
Serves	PAL learner experience	Course-master authoring screen
Node shape	No Unit/Topic levels	Full four-level hierarchy

The authoring screen's choice to read MariaDB directly is deliberate, for two reasons: the graph projection is missing the Unit/Topic levels the authoring UI needs, and a completeness-critical, teacher-facing screen should not render blank on a host where the nightly sync has not yet run. "Deployment gap" and "guarantee" are treated as different things throughout this architecture — a screen that depends on Neo4j being fully populated is a real risk unless it is designed to degrade gracefully.

8.2 Operational / admin surface — neo4j:* artisan commands

Command	Purpose
neo4j:drain	Drains queued nodes then relationships into Neo4j; runs every minute
pal:coherence-sync	Runs PAL's nightly bulk projection (eight steps)
neo4j:reconcile	Audits and can repair drift between SQL and the graph, including stale-edge cleanup
neo4j:completeness	Coverage report: SQL-vs-Neo4j row counts per label, trigger coverage, connectivity spot-checks
neo4j:export / load / verify	The Pipeline C uid-keyed batch/CSV loader

8.3 HP's own graph API

hp_erp exposes its competency/org graph directly as REST endpoints: /jobroles/{id}/graph, /organizations/{id}/graph, and /departments/{id}/graph, each returning the relevant Cypher query's nodes and relationships as JSON. This is a separate API surface from anything in next_lms_erp, backed by hp_erp's own Neo4jService.

9. Security

Connection credentials

In `next_lms_erp`, the Bolt connection and credentials are read from `config/neo4j.php`, backed by environment variables — not hardcoded. In `hp_erp`, the equivalent `NEO4J_URI` / `NEO4J_USERNAME` / `NEO4J_PASSWORD` (and the second `NEO4J_LMS_URI`) live in `.env` but are notably absent from `.env.example`, meaning a fresh setup of that repo would not surface them as a required step without prior knowledge that they exist.

Tenant isolation

`sub_institute_id` is the shared tenant identifier across all five areas — it identifies which school a record belongs to, not which product area. One inconsistency is intentional and documented in config: the native system uses `sub_institute_id = 1` as its shared/platform tenant, while PAL's own sidecar tables separately use `= 0` for the same "shared/global" meaning. Anyone writing a cross-cutting query against both native and `pal_*` tables needs to account for this explicitly rather than assume a single shared-tenant value.

Access control on graph-backed endpoints

The learner-facing Coherence Map endpoints (`/api/pal/coherence/*`) are scoped to the requesting student's own data. The authoring endpoints (`/api/lms/coherence-map*`) are scoped to course-authoring roles. Neither endpoint family exposes raw Cypher execution to the client — all graph queries are parameterized and built server-side, which is the primary injection-safety boundary for this layer; there is no user-supplied Cypher anywhere in the reviewed code.

External embed

The third-party skill-ontology viewer (Section 3.6) is loaded by `iframe` from two frontends. Because it is externally hosted and its data source is unconfirmed, it should be treated as an untrusted third-party surface for any security review — in particular, whether it receives any tenant-scoped data via query parameters or `postMessage` is worth verifying directly before relying on it for anything beyond a visual reference.

10. Current State

A brief, factual summary of what is live today versus what is designed or planned but not yet wired — useful for scoping future work accurately, without treating either state as a problem to be fixed urgently.

Capability	Status today
Curriculum sync (K12/LMS+PAL)	Live — per-row outbox every minute, nightly PAL bulk projection
Coherence Map (learner + authoring)	Live — both surfaces, both frontends
HP competency/org graph	Live — independent of next_lms_erp
PAL live diagnostic decision (ESO)	Reads SQL (learner_node_state) only; does not query Neo4j today
Misconception / corrective-content graph model	Data model exists (AFFECTS, CORRECTS_WITH edges); not yet consumed by a live recommendation feature
Graph RAG (retrieval for LLM context)	Does not exist anywhere on the platform — no vector store, no retrieval pipeline
External skill-ontology viewer relationship to HP's own graph	Unconfirmed — open question, not yet investigated end to end

Where Neo4j should not be extended by default: G2G and EB's current workloads are single-table lookups and aggregates — exactly the shape where plain SQL wins. If a genuinely deep, multi-hop traversal need appears in either area in the future (for example, a many-level org-chart walk), it is worth revisiting then, against a concrete requirement — not adopted preemptively because Neo4j already exists elsewhere on the platform.